GeoAI Roadmap Draft

Prototype → Product → World Model

English version for interview discussion

Starting from a working short-term prototype, the plan is to evolve it into an operational decision-support product and, on that trajectory, gradually extend it toward a world-model-like regional simulation layer.

What this deck is meant to show

- Ability to connect thought, design, and implementation coherently
- A decision-support workflow rather than a static analytics dashboard
- A path beyond PoC toward constraint-aware simulation and operations
- A future extension toward world-model-like regional dynamics

Guiding principles

Prototype First | **Auditability** | **Constraint-aware Simulation** | **World-Model Ready**

This prototype is not presented as a finished product. It is meant to make the architecture visible: how regional data is transformed into scenarios, how scenarios are compared and explained, and how that loop can later be extended into a dynamic simulation layer.

Phased Roadmap

Rough durations and sequencing

Phase 1 Prototype Stabilization

0–2 months

- Refine UI/UX and workflow clarity
- Stabilize spatial data input/output including GeoJSON and GeoPandas handling
- Tighten scenario comparison, explanation, and decision logging
- Ensure minimum reproducibility for demos

Phase 2 Productization

2–5 months

- Set up data pipelines and job management
- Add the operational basics: permissions, auditability, logging
- Clarify metrics and evaluation design
- Make the system usable for customer-facing PoCs

Phase 3 Constraint Simulation

5–9 months

- Add constraint-aware simulation for facility placement, mobility, and demographics
- Enable comparison across policy and investment options
- Strengthen explainability, counterfactual analysis, and sensitivity analysis
- Shift from static forecasts to dynamic comparison

Phase 4 Toward a World Model

9–15 months

- Encode regional condition into a latent state representation
- Model transitions with actions, policies, and exogenous shocks
- Support long-horizon rollout and scenario branching
- Reuse existing predictors as components rather than discard them

Phase 5 Operational Decision Platform

15+ months

- Integrate into a decision-support platform for municipalities and infrastructure operators
- Support interactive simulation and recommendation workflows
- Keep human-in-the-loop judgment explicit
- Adapt to data restrictions and closed customer environments

Static forecast → **Constraint simulation** → **World-model extension** → **Operational product**

Why move toward a World Model?

From static prediction to adaptive regional simulation

Limits of a conventional forecasting stack

- It tends to focus on one-year or one-metric prediction
- Chains of policy intervention and shock propagation are difficult to represent
- Interactions between districts or regions are under-modeled
- Policy comparison risks becoming guesswork rather than structured explanation

What a world-model direction adds

- Maintain temporal regional condition as latent dynamics
- Treat policy, investment, retreat, and reallocation as actions
- Roll out multiple futures under exogenous variables and uncertainty
- Support scenario branching for long-term consensus building

Why this fits this company context

- Population dynamics, vacant housing, facility access, and infrastructure renewal can be unified more naturally
- The outputs align directly with municipal and infrastructure decision-making
- GIS, generative AI, and simulation can be combined within one product logic
- It supports a strong loop between R&D and productization

In other words

The point is not to force a large research system into the prototype immediately. The point is to keep the current prototype aligned with a computational decomposition that can later be extended into dynamic regional state, intervention effects, and long-horizon rollout.

Transition Architecture

Keep the current app valuable while enabling the next layer

Data Layer	Public statistics / GIS / registries / administrative forms / external events
Current App Layer	Visualization / comparison / explanation / decision recording / human judgment
Predictive & Constraint Layer	Predictors / counterfactuals / constraint-aware simulation / policy evaluation
World Model Layer	State encoder / latent transition / action-conditioned rollout / branching
Decision Platform Layer	Recommendation generation / consensus support / audit logs / customer environment adaptation

Key point Do not discard the current app. Add simulation and world-model layers above it. The prototype remains useful as the interface for comparison, explanation, decision logging, and human-in-the-loop review.

Interview Framing

How to explain this roadmap clearly

How to position it in conversation

- Because the time window was short, the first goal was to make the minimum decision workflow actually run
- The next step is to stabilize data operations and add constraint-aware simulation so that PoCs can be used in real work
- From there, a world-model-like layer can be added on top to handle temporal regional state and long-horizon scenario branching
- So the current app is not the final product; it is the first scaffold connecting thought, design, and implementation

One-sentence summary

Not a static forecasting dashboard, but an initial decision-support platform that can later evolve toward a world model.

What should be evaluated

Thought | Design | Implementation | Future extension

Durations are intentionally rough. The purpose is to show sequencing and architectural direction, not a rigid delivery plan.

Implemented Design (Current Prototype)

What is already implemented in the shared app

Frontend / React + Vite

- App.jsx orchestrates state and workflow
- api.js calls FastAPI endpoints
- Components cover the map, ranking, scenario comparison, explanation, decision bar, export, and 3D viewer

Backend / FastAPI

- main.py holds API routes and core workflow logic
- Scenario generation, comparison, explanation, decision recording, and audit persistence are already connected
- The prototype intentionally keeps workflow logic close together for readability

Data Artifacts

- JSON / GeoJSON / CSV
- Region metrics, model info, regions, facilities, accessibility summaries, decisions, and audit snapshots

Build / Refresh Scripts

- Public data fetch
- Region metric rebuild
- Healthcare slice, timeline, and asset-registry generation

Current End-to-End Workflow

- Metrics calculation → scenario generation → comparison → structured explanation → agent explanation → decision record → audit / report
- The design center is not only prediction, but an explainable decision workflow

What this proves

Thought | Design | Implementation

Files reflected here: frontend/src/*, backend/app/main.py, backend/app/data/*, backend/scripts/*

Implemented Core Logic

What the current prototype already computes beyond the UI

1. Ingest / Normalize

Spatial + tabular inputs

- regions.geojson, facility JSON, accessibility summaries
- Healthcare timelines, priority profiles, 3D asset bindings
- Source metadata and refresh candidates

2. Scenario Risk Engine

Weighted risk scoring

- Baseline = aging×0.30 + vacancy×0.20 + depopulation×0.25 + medical access×0.25
- Scenario modifiers change each driver by policy focus
- Scenario intensity and total risk are recalculated per prefecture

3. Compare / Explain

Shock + uncertainty layer

- Shock library: energy / food / migration pressure
- Deterministic seed + simulated risk distribution
- Structured explanation plus optional LLM explainer over normalized payload

4. Decision Layer

Decision and audit outputs

- Decision creation with status and rationale tags
- Audit events, snapshots, and chain links
- Memo / comparison report generation for stakeholders

Why this matters

The core is not the UI look-and-feel. The real center is that regional condition is transformed into policy-aware scenarios, then closed through explanation, decision, and audit.

Bridge to world model

This deterministic core can be extended into state transition, intervention dynamics, and multi-step rollout without rewriting the decomposition from scratch.

Algorithmic Nucleus to Evolve

The minimal computational core that can later transition toward a world model

Current deterministic core

- Inputs: prefecture features, facilities / healthcare areas, accessibility and timeline artifacts
- Operators: scenario-specific modifier maps, weighted score aggregation, shock profile generation, simulated risk distribution, structured explanation builder
- Outputs: ranked regions, scenario deltas, recommendation reasons, auditable decision objects

Missing dynamic layer

- State transition over time
- Intervention effects and delayed responses
- Cross-region interaction and spillover
- Multi-step rollout under constraints

World-model-ready formulation

- State: regional latent state + infrastructure / demographic context
- Action: medical reinforcement, housing reuse, compact-city policy, budget allocation
- Transition: next-state evolution under shocks, demand shifts, and network effects
- Readout: long-horizon scenario tree, policy value, explanation, and robust planning

Message for interview

This prototype already exposes the right decomposition — data/state, scenario operator, explanation, decision, and audit. The next step is not a rewrite, but a dynamical extension.